Developer Evaluation Report

Sidali Bedrani — Mobile Developer (Flutter / BLE)

Period: May 1 – June 22, 2026 | Prepared: June 22, 2026 | Wergonic

---

1. Scope & Methodology

This evaluation covers the period May 1 – June 22, 2026. It is based on three data sources: (1) task records exported from the project management tool, including logged hours and task IDs; (2) the full Git history of the **wergonic-flutter** repository (all branches, primarily *dev* and *new_refactoring*); (3) commit diffs measuring the actual code changes per task. Tasks were classified as *research/investigation* (no code deliverable expected) or *coding* (a commit is expected). Each coding task ID was cross-referenced against commit messages.

It is important to note that much of the coding work in this period was done using Claude Code (AI-assisted coding). This is an approved workflow. However, it changes the expected time-to-output ratio: tasks that would normally take a developer several hours of manual coding can now be completed in significantly less time. Time estimates should reflect the actual work performed — prompting, reviewing AI output, testing on hardware — not what the task would have taken without AI assistance.

2. Summary Numbers

Metric	Value
Employment	Full-time, 40 h/week
Expected hours (leave-adjusted)	304 h (2 sick days taken)
Logged hours	229.95 h
Utilization	75.6 %
Total tasks	~88
Research / investigation tasks	~24 tasks — 87.5 h (38 % of total)
Coding tasks	~64 tasks — 142.5 h
Non-merge commits	75

3. Commit Coverage

The majority of coding tasks have properly tagged commits (e.g. `fix: [WH-296]`). This is good practice and makes traceability straightforward. However, the following coding tasks have **no identifiable commit** on any branch:

Task	Hours	Date	Description
WH-218	4 h	May 7	fix cascading by pausing stream
WH-219	3 h	May 7	change pause by widening radio bandwidth
WH-223	4 h	May 8	optimize the pause to reduce data loss
WH-224	2 h	May 8	structural pause (noted: broke the flow)
WH-225	1 h	May 8	cubit-side paused flag
WH-226	3 h	May 8	reduce data loss to 3 s

Task	Hours	Date	Description
WH-336	3 h	Jun 3	make RSSI live in pairing/calibration/recording
WH-337	1 h	Jun 3	fix live RSSI interfering with battery
WH-207	2 h	May 6	guard isWaitingForAdapter reset

Total: 23 hours of coding work with no commit. The May 7–8 block (17 h across 6 tasks) is the most notable gap — two full working days of reported feature/bug work without a single commit on any branch. Even exploratory or failed attempts should normally produce at least a WIP commit.

Note: WH-336/337 may have been rolled into the WH-335 commit from June 5. WH-448 and WH-454 (2.5 h, June 19) are recent and in Testing status — they may be work-in-progress not yet pushed.

4. Time vs. Code Output

Some tasks show a significant gap between logged hours and the size of the actual code change. While BLE/hardware work involves build-deploy-test cycles that don't show up in diffs, the following stand out — particularly in an AI-assisted workflow where code generation itself is fast:

Task	Logged	Change Size	Notes
WH-296	3 h	10 lines	Stuck connecting status fix — small logic change
WH-297	3 h	22 lines	Reconnect delay — minor parameter adjustment
WH-251	3 h	31 lines	Calibration reload fix
WH-246	3 h	~10 lines (+pubspec.lock)	Calibration disconnect — actual code minimal
WH-264	4 h	68 lines	Skip wrist calibration — single conditional block
WH-389	2 h	11 lines	Stream resubscribe after reconnect

These tasks average roughly 1 h of realistic effort each (prompt, review, deploy to device, verify). The logged times suggest the estimates were not adjusted for AI-assisted development speed.

5. Research & Investigation Time

38 % of all logged time (87.5 h) was spent on research or investigation tasks. These produce no code artifact and are inherently difficult to verify. Some level of research is expected and necessary in BLE/embedded work — debugging on physical hardware, analyzing sensor logs, reverse-engineering SDKs. However, the proportion is notably high.

Task	Hours	Description
WH-418	13 h	Verify CSV expected_rows vs rows_written
WH-313	11 h	Check why hand/forearm disconnect and don't reconnect
WH-18 (parent)	10.25 h	CSV/ZIP file verification (on top of 10.25 h coding)
WH-390	6 h	Hard test the native BLE migration
WH-243	5 h	Full test round
WH-316 + WH-314	8 h	Investigate MDS stream disconnects (two sub-tasks, same issue)
WH-332	4 h	Check code for non-reconnection triggers
WH-327	4 h	Full test round before pushing

A typical ratio for this type of work would be 20–25 % research vs. coding. Going forward, research tasks above 4 h should include a written summary of findings, even if brief, to provide traceability and demonstrate the

investigation was thorough.

6. Strengths

- **BLE domain expertise** — The native Kotlin BLE engine migration (WH-349 and sub-tasks) was the most complex initiative in this period. It required reverse-engineering a closed-source AAR, designing a hybrid architecture, and migrating scanning, connection, streaming, battery telemetry, RSSI, and DFU flows. This demonstrates real depth.
- **Commit discipline** — Task IDs are consistently tagged in commit messages. The PR flow through dev/new_refactoring is clean. This makes code review and traceability straightforward.
- **Output volume** — 75 non-merge commits across ~7 weeks. The RSSI feature (WH-335 family) and native migration were both delivered end-to-end.
- **Progressive complexity** — Work progressed from BLE bug fixes to a new feature (RSSI monitoring) to a full architectural migration (native engine). This shows growth over the period.
- **Hardware-validated work** — BLE sensor work can't be faked. Pairing, calibration, and recording either work on real sensors or they don't. The deployed tasks passed real-device testing.

7. Areas for Improvement

- **Time estimates must reflect AI-assisted speed.** When Claude Code generates the code, the developer's work is prompting, reviewing, and testing — not writing every line manually. Logged hours should match this reality. A 10-line fix should not take 3 hours regardless of the tooling used for the fix itself.
- **Every coding task needs a commit.** 23 hours of coding work had no corresponding commit on any branch. Even failed experiments or iterative approaches should be committed (on a feature branch) so that work is visible and reviewable. The May 7–8 gap of 17 hours is the clearest example.
- **Research tasks need deliverables.** At 38 % of total time, investigation tasks are a large investment. Tasks above 4 h should produce a brief written finding — a comment on the task, a markdown file, or structured notes. This helps the team and makes the time investment verifiable.
- **Utilization gap.** 229.95 h logged against 304 h expected (75.6 %). The remaining ~74 h (over 9 working days) are unaccounted for beyond 2 sick days. Time tracking should cover the full working week, including meetings, code reviews, and other activities.
- **Task descriptions should be in your own words.** Several task descriptions — particularly in the native migration — read as AI-generated text. While using AI tools for coding is encouraged, task descriptions should reflect the developer's own understanding in plain language. This helps the team assess comprehension, not just output.

8. Expectations Going Forward

1. All coding tasks must have at least one commit, tagged with the task ID. No exceptions.

2. Time logs should be adjusted to reflect AI-assisted workflows. If Claude Code generates the code in minutes, log the actual time spent: prompting, reviewing, testing, iterating.

3. Research/investigation tasks above 4 hours must include a written summary of findings attached to the task.

4. Full-time hours (40 h/week) should be accounted for. If time is spent on activities not captured in tasks (meetings, reviews, setup), those should be logged separately.

5. Task descriptions should be written by the developer, not generated by AI tools.
